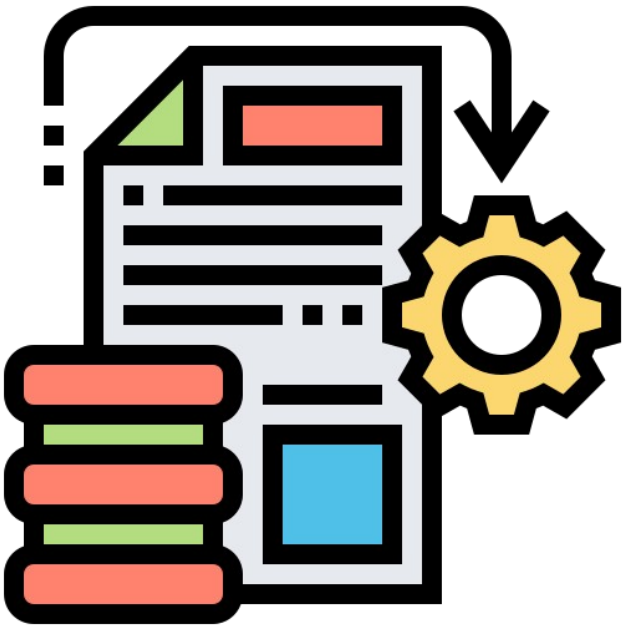
Institut Notre-Dame Beauraing

UAA10

Le traitement de l'information



Technique de transition

–

Option Informatique

–

Titulaire du cours : L. Embrechts

Quelques questions de réflexions...

1. A l'heure actuelle, pensez-vous qu'un ordinateur puisse penser comme un être humain ? Et pourquoi ? (Basez-vous vos arguments sur quelconque réalités/actualités?)

2. Pouvez-vous expliquer comment physiquement, un fichier est stocké sur l'ordinateur ?

3. Vous possédez une image en format numérique (.jpg) pouvez-vous expliquer comment, depuis le fichier source, celle-ci s'affiche à l'écran ?

4. Vos parents vous proposent un nouveau logiciel. Il en existe deux versions, la version 32 bits et la version 64bits. Laquelle choisissez vous ? Et pourquoi ?

5. Quelle est la différence entre un nombre entier et un nombre flottant ?
   Même question mais pour l'informatique, quelle sera la différence dans un ordinateur entre un nombre entier et un nombre flottant.

Introduction

1 Traitement formel, de quoi s'agit-il ?

1.1 Traitement formel et informel de l'information

Le traitement de l'information par un ordinateur est purement formel. C'est à dire que l'on réduit la définition de l'information à son aspect syntaxique. Bien que que certains algorithmes bien étudiés puissent donner l'impression contraire. L'expression sémantique est artificielle et limitée. L'information est donc considérée telle une suite de symboles dénuée de sens.

L'ordinateur est une machine à traiter les informations. Ces traitements sont limités aux traitement purement formels. L'étudiant qui perçoit bien cette énorme limite du traitement informatique peut éviter de nombreux pièges et porter sur les produits qu'il utilise un regard critique quand à leur conception.

Voici des exemples permettant d'illustrer la différence entre traitements formels et non formels.

Si on demande à une personne de remplacer le point d'interrogation par une lettre dans les deux mots suivants :

CH?T

CH?EN

ça ne lui pose pas problème (pour autant qu'elle connaisse la langue française) et dans la mesure où elle donne à cette question une interprétation plus poussée que de simplement mettre une lettre, n'importe laquelle, à la place du point d'interrogation. Pour trouver la réponse, cette personne (être humain) n'effectue pas un traitement formel.

Si on lui pose la même question, et si elle veut résoudre le problème de la même façon dans les cas suivants :

A ?GYAL O ?DOG

ça lui pose problème si elle ne connaît pas la langue dans laquelle ces mots ont du sens. Elle ne peut donner de réponse à moins de disposer d'un dictionnaire lexicographique de cette langue. Le traitement prendra alors plus de temps et la réponse résultera d'un traitement formel de ces informations.

1.2 Exercice pratique : Traitements formels et informels

1. Traitement formel et informel de l'information : Si on vous demande, à vous être humain, les anagrammes du mot CRANE, que proposerez vous comme réponse ? Que serait capable de faire un « ordinateur » face à la même situation ?

2. **Dans un traitement sans intelligence artificielle**, dans les énoncés suivants quels sont ceux qui sont faciles à formaliser ? Moyennement difficiles à formaliser ? pratiquement impossibles à formaliser ? Argumentez votre réponse.

Compter le nombre de formes verbales dans un document	
Supprimer dans un document toutes les parties de texte entre parenthèses	
Traduire une notice explicative	
Trouver quel jour de la semaine est le 2 octobre sans avoir de calendrier sous la main	
Remplir une grille de mots croisés si les définition sont fournies	



Ces exemples permettent à l'utilisateur de deviner les traitements possibles d'un correcteur orthographique et parfois, ses dysfonctionnements. Ils donnent une idée plus claire de la manière dont ces traitements sont effectués et battent en brèche l'idée d'ordinateur intelligent véhiculée par d'importantes couches logicielles.

En traitement de texte, par exemple, on peut s'intéresser à la définition de mot, de phrase, de paragraphe, de page,... et faire découvrir le concept de délimiteur. Dans l'utilisation du tableur, l'utilisateur peut comprendre que la reconnaissance des types de données soit implicite (comme elle peut l'être dans certains langages de programmation). Avec le gestionnaire de fichiers il ne s'étonnera pas, dans une base de données, de l'absence d'enregistrement répondant au critère : nom = DUPONT alors qu'il est certain d'avoir encodé les renseignements concernant Monsieur Dupont...
Les exemples qui trouvent des explications dans le formalisme des traitements sont nombreux et quotidiens. Ainsi, pourquoi certains langages ou logiciels acceptent-ils des commandes écrites en majuscules ou en minuscules ?

En ayant pleine conscience de ces concepts, l'utilisateur sera à même de comprendre les exigences d'une syntaxe parfois rigoureuse et pourra apprécier le travail de fond du concepteur lorsque ces rigueurs syntaxiques ont été aplanies.

1.3 Traitement formel de l'information : idées maîtresses.

Trois aspects du traitement formel de l'information sont à mettre en évidence. Ils sont fondamentaux et leur mauvaise compréhension empêche les utilisateurs de résoudre un nombre considérable de problèmes rencontrés au cours d'une session de travail. Nous les énonçons de la manière suivante :

> *Pour être traitable par un ordinateur, toute information doit être numérisée.*

> *Pour être possible, tout traitement d'une information par un « ordinateur » doit être décrit de manière purement formelle.*

Ce n'est pas à l'ordinateur que l'utilisateur est confronté, mais au couple indissociable « ordinateur-logiciel » (« hardware-software »), un exécutant sans cesse en évolution au cours d'une même session de travail.

Ces principes justifient que le mot ordinateur signifie dans ce contexte une machine capable de traiter des informations de manière formelle pour autant qu'on lui ait préalablement indiqué comment mener à bien ce traitement.
La bonne compréhension de ces principes est de nature à augmenter l'autonomie des utilisateurs :

- Ils doivent appréhender de manière plus correcte l'évolution technologique tant logicielle que matérielle.
- Ils doivent être capable d'anticipation concernant les capacités attendues des programmes.
- Ils doivent manifester des capacités d'adaptation envers le passage d'une version à une autre, d'un logiciel spécifique à un autre du même type, d'un type de produit à un autre, voire d'un type d'ordinateur à l'autre.







Les supports type disquettes, floppies, clés USB, cartes SD, disques dur internes fonctionnent tous de la même manière depuis le début de leur existence. Seuls leur vitesse de transfert et leur capacité ont augmenté au fil du temps.

Ces support digitaux, fonctionnent avec un code binaire. C'est à dire qu'ils ne supportent que deux symboles. Impossible donc pour eux d'encoder notre alphabet complet.

Mais alors, comment transformeriez-vous un message de votre langue (qui possède plus de 26 caractères et tous ses symboles) en une suite de deux symboles ? De quoi avez-vous besoin pour que cela soit compréhensible pour quelqu'un d'autre ?

1.4 Exercice pratique : la communication par code est universelle.

Par groupe de 2 :

- ✔ Inventez un code
- ✔ Codez une question type « Qui es-tu? », « D'où viens-tu ? ».
- ✔ Rédigez un mode d'emploi du code avec rigueur.

Contraintes :

- Le message doit être transformé en **x** et □ ;
- Les signes **x** et □ doivent se suivre sans aucun espace ;
- Le message doit être accompagné d'un mode d'emploi pour permettre à un autre groupe de le déchiffrer ;
- Le mode d'emploi doit être clair et complet ;
- Le message doit contenir au moins deux espace et un point d'interrogation

Questions :

- Parmi les messages et modes d'emploi réalisés, lesquels sont décryptables ?
- Pour chaque message qui n'a pas pu être décrypté, indiquez ce qui a posé problème au décodage? Pourquoi cela vous a-t-il posé problème ?
- Dans les messages décryptés, le sont-ils tous sans intelligence, sans supposition ? En d'autres termes, une machine (sans IA) pourrait-elle les décrypter ? Sinon, pourquoi ?

1.5 La communication par code idéale

L'objectif est de trouver une codification universelle, rigoureuse et structurée : la codification avec des suite de **x** et de □ sur une longueur fixe.

Une suite d'un seul symbole offre deux possibilités : **x** et □
Une suite de 2 symboles offre quatre possibilités : **□ □, □ x, x □, x x**
Une suite de 3 symboles offre 8 possibilités : **□ □ □, □ □ x, □ x □, □ x x, x □ □, x □ x, x x □, x x x**
Une suite de 4 symboles offre 16 possibilités : **...**

Bien entendu, il faut tenir compte du système complet. C'est à dire des lettres majuscules, minuscules, des chiffres, des symboles de ponctuation, des signes d'opération. Nous arrivons donc à un total d'environ 120 caractères soit 2^7 = 128 possibilités.

En informatique, quelle que soit la nature de l'information traitée par un ordinateur (image, son, texte, vidéo), elle l'est toujours sous la forme d'un ensemble de nombres écrits en base 2. Cela provient du fait que physiquement, cette information sous forme binaire peut se représenter par un signal électrique ou magnétique, qui au-delà d'un certain seuil, correspond à la valeur 1, en deçà, correspond à la valeur 0. Les symbole **x** et □ sont comparables aux états électriques/magnétiques 0 et 1. Ils sont en d'autre termes les chiffres binaires.

... Comprenez ici que vous venez de définir le code ASCII.

Ajoutez à celui-ci, les caractères accentués. Une suite de 16 symboles sera alors nécessaire pour le codage du système complet. Le code Unicode, sur 16 bits regroupe la quasi totalité des alphabets existants (arabe, arménien, cyrillique, hébreu, grec, latin...) et est compatible avec le code ASCII.

2 Le codage des nombres

2.1 La base décimale, binaire ou hexadécimale (conversions de bases)

Nous utilisons le système décimal (base 10) dans nos activités quotidiennes. Ce système est basé sur dix symboles, de 0 à 9, avec une unité supérieure (dizaine, centaine, etc.) à chaque fois que dix unités sont comptabilisées. C'est un système positionnel, c'est-à-dire que l'endroit où se trouve le symbole définit sa valeur. Ainsi, le 2 de 523 n'a pas la même valeur que le 2 de 132. En fait, 523 est l'abréviation de $5x10^2 + 2x10^1 + 3x10^0$. On peut selon ce principe imaginer une infinité de systèmes numériques fondés sur des bases différentes.
En informatique, outre la base 10, on utilise très fréquemment le système binaire (base 2) puisque l'algèbre booléenne est à la base de l'électronique numérique. Deux symboles suffisent : 0 et 1.
On utilise aussi très souvent le système hexadécimal (base 16) du fait de sa simplicité d'utilisation et de représentation pour les mots machines (il est bien plus simple d'utilisation que le binaire). Il faut alors six symboles supplémentaires : A (qui représente le 10), B (11), C (12), D (13), E (14) et F (15).

Le tableau ci-dessous montre la représentation des nombres de 0 à 15 dans les bases 10, 2 et 16.

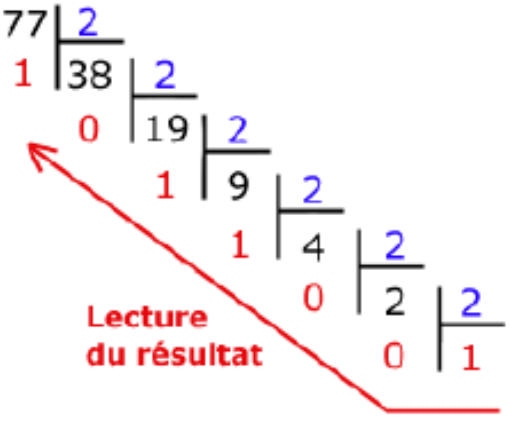
Décimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Binaire	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Hexadécimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Conversion décimal - binaire

Convertissons 01001101 en décimal à l'aide du schéma ci-dessous :

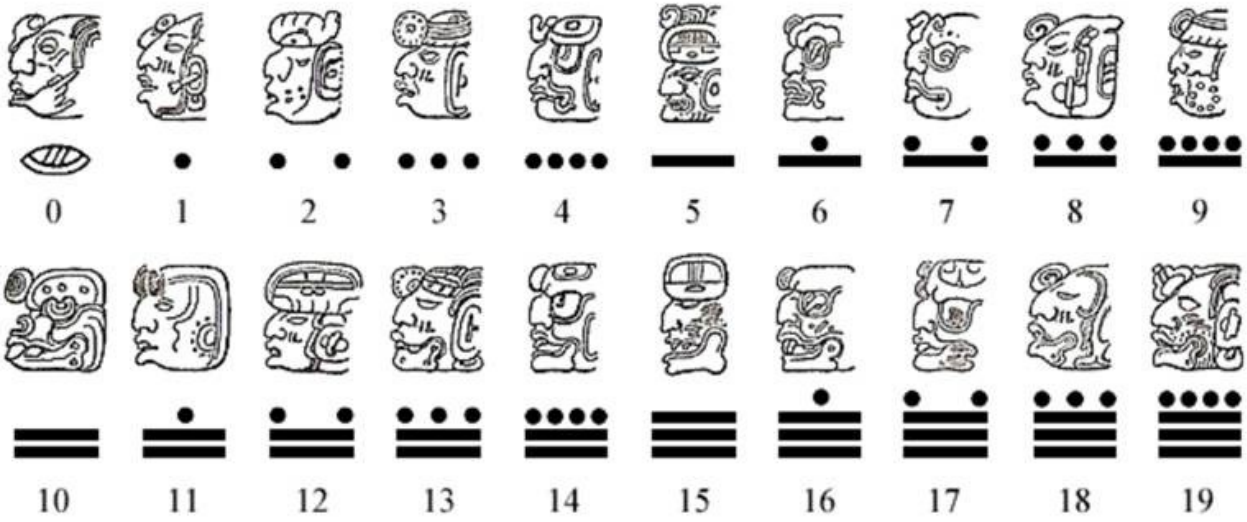
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	0	1	1	0	1

Le nombre en base 10 est $2^6 + 2^3 + 2^2 + 2^0 = 64 + 8 + 4 + 1 = 77$.
Allons maintenant dans l'autre sens et écrivons 77 en base 2. Il s'agit de faire une suite de divisions euclidiennes par 2. Le résultat sera la juxtaposition des restes.
77 s'écrit donc en base 2 : 1001101.
Si on l'écrit sur un octet, cela donne : 01001101.

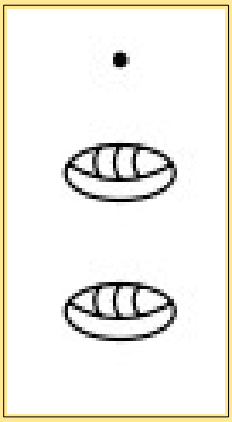


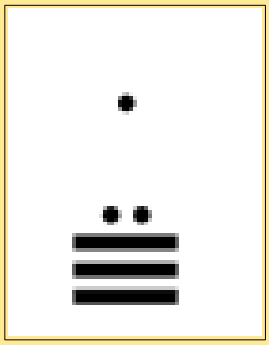
2.2 Autres bases (conversions de bases)

Voici le système vicésimal des Mayas



Selon leur système numérique.
Que valent ces nombres en décimal ?





2.3 Représentation des nombres entiers

Représentation d'un entier naturel

Un entier naturel est un nombre entier positif ou nul. Le choix à faire (c'est-à-dire le nombre de bits à utiliser) dépend de la fourchette des nombres que l'on désire utiliser. Pour coder des nombres entiers naturels compris entre 0 et 255, il nous suffira de 8 bits (un octet) car $2^8 = 256$.
D'une manière générale un codage sur n bits pourra permettre de représenter des nombres entiers naturels compris entre 0 et 2^n-1.

Exemples : $9 = 00000101_2$ $128 = 10000000_2$ etc.

Représentation d'un entier relatif

Un entier relatif est un entier pouvant être négatif. Il faut donc coder le nombre de telle façon que l'on puisse savoir s'il s'agit d'un nombre positif ou d'un nombre négatif, et il faut de plus que les règles d'addition soient conservées. L'astuce consiste à utiliser un codage que l'on appelle complément à deux. Cette représentation permet d'effectuer les opérations arithmétiques usuelles naturellement.

- **Un entier relatif positif ou nul** sera représenté en binaire (base 2) comme un entier naturel, à la seule différence que le bit de poids fort (le bit situé à l'extrême gauche) représente le signe. Il faut donc s'assurer pour un entier positif ou nul qu'il est à zéro (0 correspond à un signe positif, 1 à un signe négatif). Ainsi, si on code un entier naturel sur 4 bits, le nombre le plus grand sera 0111 (c'est-à-dire 7 en base décimale).
  - Sur 8 bits (1 octet), l'intervalle de codage est [-128, 127].
  - Sur 16 bits (2 octets), l'intervalle de codage est [-32768, 32767].
  - Sur 32 bits (4 octets), l'intervalle de codage est [-2147483648, 2147483647].

  D'une manière générale le plus grand entier relatif positif codé sur n bits sera 2n-1-1.

- **Un entier relatif négatif** sera représenté grâce au codage en complément à deux.

Principe du complément à deux

1. Écrire la valeur absolue du nombre en base 2. Le bit de poids fort doit être égal à 0.
2. Inverser les bits : les 0 deviennent des 1 et vice versa. On fait ce qu'on appelle le complément à un.

3. On ajoute 1 au résultat (les dépassements sont ignorés).

Cette opération correspond au calcul de 2^n-|x|, où n est la longueur de la représentation et |x| la valeur absolue du nombre à coder.
Ainsi -1 s'écrit comme 256 - 1 = 255 = 11111111_2, pour les nombres sur 8 bits.

Exemple
On désire coder la valeur -19 sur 8 bits. Il suffit :

1. d'écrire 19 en binaire : 00010011
2. d'écrire son complément à 1 : 11101100
3. et d'ajouter 1 : 11101101

La représentation binaire de -19 sur 8 bits est donc 11101101.
On remarquera qu'en additionnant un nombre et son complément à deux on obtient 0. En effet,
00010011 + 11101101 = 00000000 (avec une retenue de 1 qui est éliminée).

Exercice 1 :

Codez les entiers relatifs suivants sur 8 bits (16 si nécessaire) : 456, -1, -56, -5642.

Exercice 2 :

Que valent en base dix les trois entiers relatifs suivants ?

01101100

11101101

1010101010101010

2.4 Représentation des nombres réels

En base 10, l'expression 652,375 est une manière abrégée d'écrire :

$$6 \cdot 10^2 + 5 \cdot 10^1 + 2 \cdot 10^0 + 3 \cdot 10^{-1} + 7 \cdot 10^{-2} + 5 \cdot 10^{-3}.$$

Il en va de même pour la base 2. Ainsi, l'expression 110,101 signifie :

$$1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}.$$

Conversion de binaire en décimal

On peut ainsi facilement convertir un nombre réel de la base 2 vers la base 10. Par exemple :

$$110{,}101_2 = 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 4 + 2 + 0{,}5 + 0{,}125 = 6{,}625_{10}.$$

Exercice 1

Transformez $0{,}0101010101_2$ en base 10.

Transformez $11100{,}10001_2$ en base 10.

Conversion de décimal en binaire

Le passage de base 10 en base 2 est plus subtil. Par exemple : convertissons 1234,347 en base 2.

- La partie entière se transforme comme telle que $1234_{10} = 10011010010_2$
- On transforme la partie décimale selon le schéma suivant :

0,347·2 = **0**,694	0,347 = 0,**0**...
0,694·2 = **1**,388	0,347 = 0,0**1**...
0,388·2 = **0**,766	0,347 = 0,01**0**...
0,766·2 = **1**,552	0,347 = 0,010**1**...
0,552·2 = **1**,104	0.347 = 0,0101**1**...
0,104·2 = **0**,208	0,347 = 0,01011**0**...
0,208·2 = **0**,416	0,347 = 0,010110**0**...
0,416·2 = **0**,832	0,347 = 0,0101100**0**...
0,832·2 = **1**,664	0,347 = 0,01011000**1**...
0,664·2 = **1**,328	0,347 = 0,01011000**1**...
0,328·2 = **0**,656	0,347 = 0,010110001**0**...

On continue ainsi jusqu'à la précision désirée...



Attention ! Un nombre à développement décimal fini en base 10 ne l'est pas forcément en base 2. Cela peut engendrer de mauvaises surprises.

Le 25 février 1991, pendant la Guerre du Golfe, une batterie américaine de missiles Patriot, à Dharan (Arabie Saoudite), a échoué dans l'interception d'un missile Scud irakien. Le Scud a frappé un baraquement de l'armée américaine et a tué 28 soldats. La commission d'enquête a conclu à un calcul incorrect du temps de parcours, dû à un problème d'arrondi. Les nombres étaient représentés en virgule fixe sur 24 bits. Le temps était compté par l'horloge interne du système en dixième de seconde. Malheureusement, 1/10 n'a pas d'écriture finie dans le système binaire : 1/10 = 0,1 (dans le système décimal) = 0,0001100110011001100110011... (dans le système binaire). L'ordinateur de bord arrondissait 1/10 à 24 chiffres, d'où une petite erreur dans le décompte du temps pour chaque 1/10 de seconde. Au moment de l'attaque, la batterie de missile Patriot était allumée depuis environ 100 heures, ce qui a entraîné une accumulation des erreurs d'arrondi de 0,34 s. Pendant ce temps, un missile Scud parcourt environ 500 m, ce qui explique que le Patriot soit passé à côté de sa cible.

Conversion hexadécimal - binaire

Pour convertir un nombre binaire en hexadécimal, il suffit de faire des groupes de quatre bits (en commençant depuis la droite). Par exemple, convertissons 1001101 :

Binaire	0100	1101
Hexadécimal	4	D

1001101 s'écrit donc en base 16 : 4D.
Pour convertir d'hexadécimal en binaire, il suffit de lire ce tableau de bas en haut.

Exercice 1

Donnez la méthode pour passer de la base décimale à la base hexadécimale (dans les deux sens).

Exercice 2

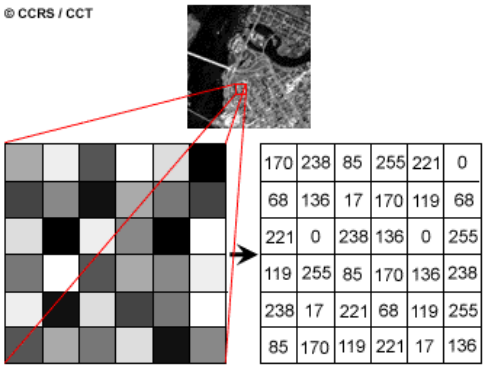
Complétez le tableau ci-dessous. L'indice indique la base dans laquelle le nombre est écrit.

	Bases		
	2	10	16
1001010110_2			
2002_{10}			
$A1C4_{16}$			

Exercice 3

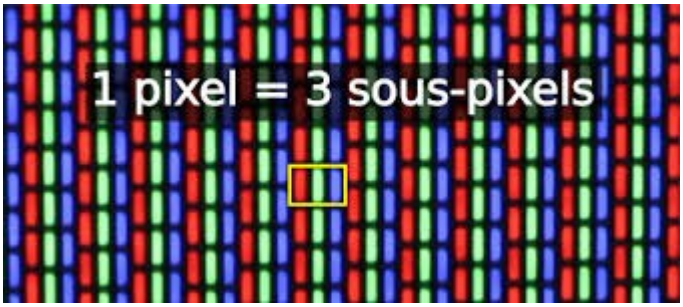
Écrivez en Javascript un programme permettant de convertir un nombre d'une base de départ d vers une base d'arrivée a (d et a compris entre 2 et 16).

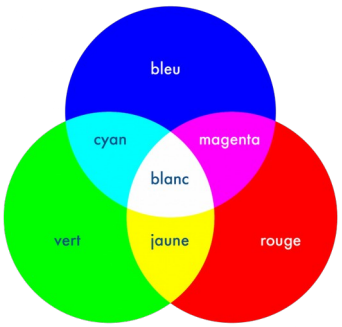
2.5 Le codage des images



Une image noire et blanche est composée d'un tableau en deux dimensions de valeurs allant de 0 à 255. Il y a une valeur pour chaque pixel de l'image. La valeur est équivalente à l'intensité de lumière qui sera diffusée dans la led de ce pixel sur l'écran. Ainsi une valeur à 0 ne diffuse pas de lumière et représente un point noir à l'écran.

Une image colorée est composée d'un tableau à deux dimensions dont les valeurs sont un ensemble de 3 nombres allant de 0 à 255. Chacun de ces nombres représente l'intensité de lumière à émettre sur une led rouge, verte, bleue. En adéquation avec la théorie de la lumière (synthèse additive), ces 3 sous-pixels formeront la couleur du pixel.





Souvent sur le web, au lieu d'expliciter les 3 valeurs, la couleur est explicité en un nombre hexadécimal.

Exemple #9A547F → 9A, 54, 7F → les 3 valeurs notées en hexadécimal.

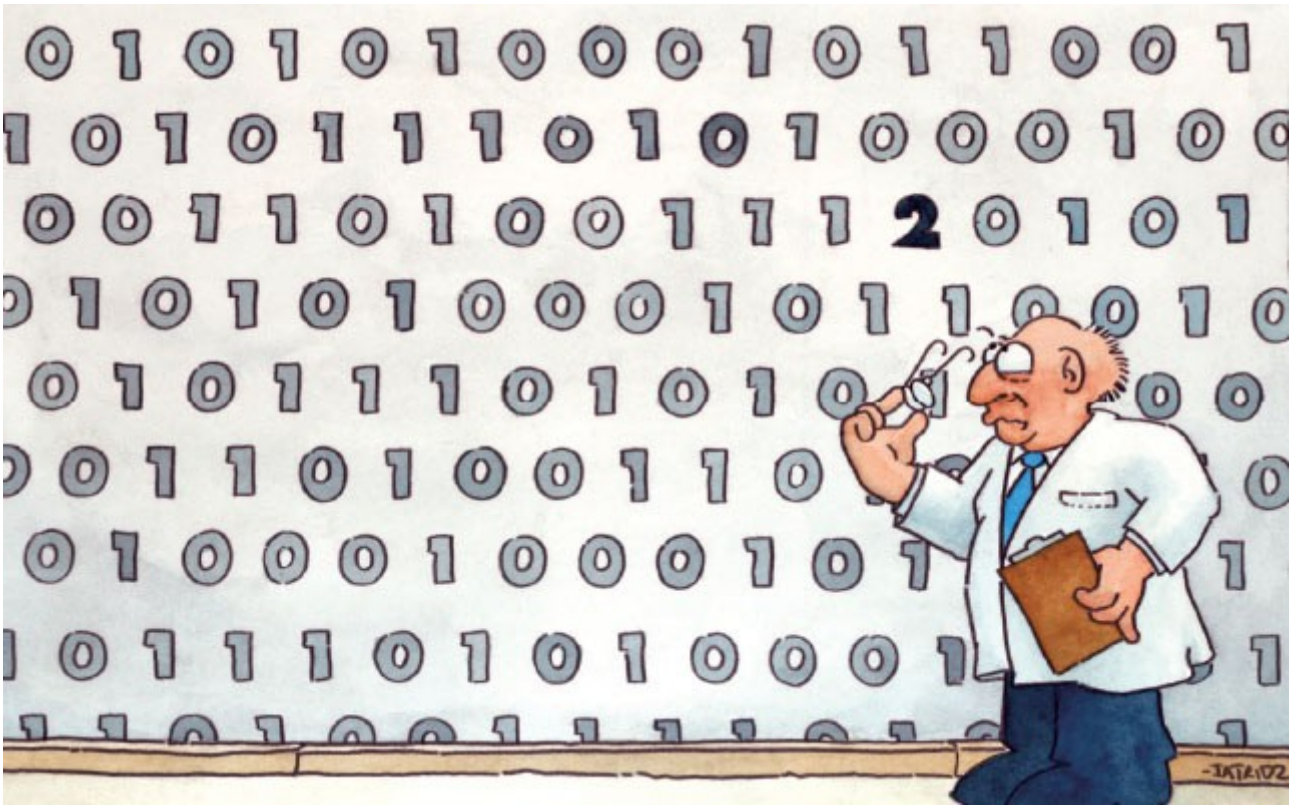
1. #957A2B. Que vaut cette couleur notée en hexadécimal,
   - En nombre décimal,
   - En binaire ?
2. Combien d'octets sont nécessaire à la l'écriture d'un pixel ?
3. Pour la conversion en binaire quel modèle de conversion avez-vous utilisé ? Pourquoi celui-ci par rapport aux autres ?

4. Si ces 3 valeurs étaient encodées selon la norme IEEE 754, combien d'octets seraient nécessaires ?

5. Comment serait écrite la valeur Rouge selon la norme IEEE754 ?

6. S'agit-il d'un traitement formel ou informel de l'information ? Expliquez pourquoi en illustrant vos propos d'exemples concrets.

3 Codes détecteurs et correcteurs d'erreurs



Un code correcteur est une technique de codage basée sur la redondance. Elle est destinée à corriger les erreurs de transmission d'un message sur une voie de communication peu fiable.

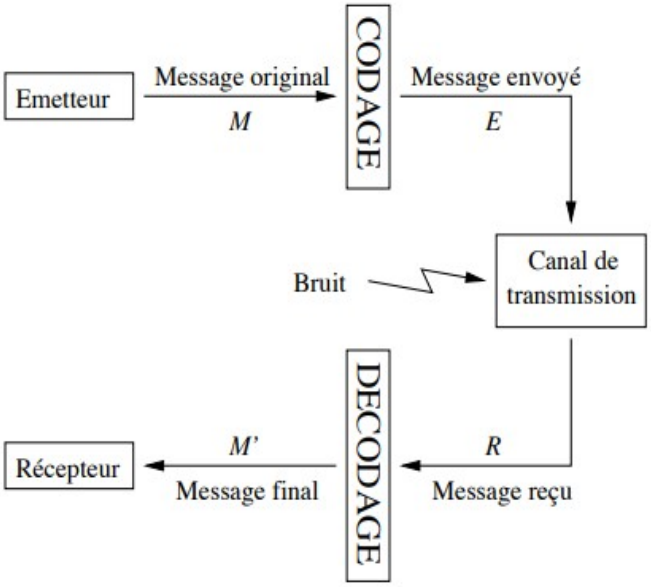
La théorie des codes d'erreurs ne se limite pas qu'aux communications classiques (radio, câbles, fibre, ...) mais également aux supports de stockage comme les disques compacts, la mémoire RAM et d'autres application où l'intégrité des données est importante.

Pourquoi ces techniques de détection d'erreurs et de correction codage existent-elles?

Ls canaux de trasmissian sont impazfaits !

Par exemple : la probabilité d'erreur sur un ligne de téléphone est de 10^{-7} , cela peut même atteindre $10^{-4.}$
Avec un taux d'erreur à 10-6 et un connexion à 1Mo/s => 8bits seront erronés par seconde.

3.2 Schéma général de communication.



- Le message à transmettre M est d'abord codé en un message E. Le codage introduit un supplément d'information (redondances).
- Le message E est envoyé sur le canal de transmission.
- A cause de l'imperfection du canal, le message reçu R n'est pas forcément identique au message envoyé E : il peut contenir des erreurs.
- Le message reçu R est ensuite décodé : des erreurs éventuelles peuvent être détectées et corrigées.
- Le message décodé M' est identique au message d'origine M si il n'y a pas eu trop d'erreurs dans la transmission.

3.3 Principe général :

Chaque suite de bits à transmettre est augmentée par une autre suite de bits dite « de redondance » ou « de contrôle ».

Pour chaque suite de **k** bits transmise, on ajoute **r** bits.
On dit alors que l'on utilise un code **C(n,k)** pour **n = k+r**

```
A la réception, les bits ajoutés permettent d’effectuer des contrôles.
```

3.4 Exemples de la vie courante.

Le premier système de code détecteur d'erreur est la **checksum.** Cette technique compose souvent la base de système de détection d'erreurs plus complexes que nous verrons plus loin.

Il en existe bien d'autres, dans des systèmes plus large que juste autour du binaire.

En voici trois exemple :

- le numéro de compte IBAN,
- Le code CPP,
- le code ISBN.

Le numéro de compte bancaire belge :

BE	2 chiffres	12 chiffres
Code du pays	*Clé de contrôle*	*Numéro de compte*

En Belgique, l'IBAN est formé de 16 caractères : il se compose d'abord des lettres "BE" suivies de 2 chiffres, puis des 12 chiffres du numéro de compte national.

Par exemple : le numéro de compte belge

`539-0075470-34`

devient

`BE68 5390 0754 7034.`

Ainsi, l'algorithme pour vérifier un numéro IBAN est le suivant :

1. Déplacer les 4 premiers caractères à la fin du compte
2. Remplacer les lettres par des chiffres au moyen d'une table de conversion (A=10, B=11, C=12 etc.)
3. Appliquer le modulo 97 à ce nombre
4. Si le résultat donne 1, le numéro est correct

Testez avec votre propre numéro de compte !

Enoncé :
Alice souhaite rembourser Thomas via virement bancaire. Celui-ci lui a indiqué différents numéro de comptes sur un extrait de papier. Lequel est correct ?

`BE41 0630 1234 5610`

`BE71 0961 2335 6769`

`BE68 5390 0754 7034`

3.5 Distance de Hamming

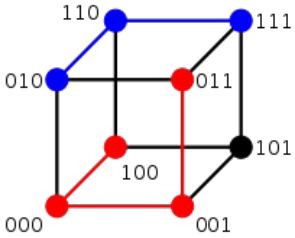
La distance de Hamming doit son nom à Richard Hamming. Il s'agit d'une notion mathématique utilisée en informatique, en traitement du signal et dans les télécommunications. Elle joue un rôle important dans la théorie des codes correcteurs.

La distance de Hamming permet de quantifier la différence entre deux séquences de symboles. À deux suites de symboles de même longueur, elle associe le nombre de positions où les deux suites diffèrent. C'est une distance au sens mathématique du terme.

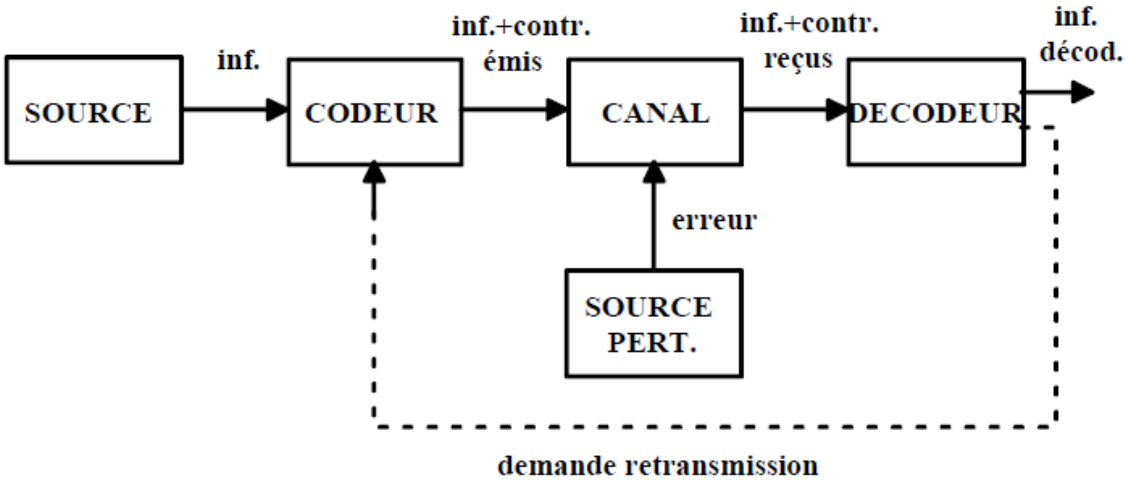
Elle est notamment utilisée en télécommunication pour compter le nombre de bits altérés dans la transmission d'un message d'une longueur donnée.

Exemples :

- ➔ La distance de Hamming entre 1011101 et 1001001 est 2.
- ➔ La distance de Hamming entre 2143896 et 2233796 est 3.
- ➔ La distance de Hamming entre "ramer" et "cases" est 3.



3.6 Exercice de synthèse



Nous sommes en pleine ère numérique. Qu'est-ce que cela veut dire ? Tout simplement qu'une partie énorme des informations échangées à travers la planète est matériellement représentée sous la forme de nombres.

La surface d'un CD est fragile, mais un code correcteur est utilisé lors de sa gravure. Messages électroniques, téléphonie mobile, transactions bancaires, téléguidage de satellites, télétransmission d'images, disques CD ou DVD, etc. : dans tous ces exemples, l'information est traduite en suites de nombres entiers, correspondant physiquement à des signaux électriques ou autres. Plus précisément même, l'information est codée sous forme de suites de chiffres binaires - des 0 ou des 1, appelés aussi bits. Par exemple, dans le code ASCII (American Standard Code for Information Interchange) utilisé par les micro-ordinateurs, le A majuscule est codé par l'octet (séquence de 8 bits) 01000001, le B majuscule par 01000010, etc.

Un problème majeur de la transmission de l'information est celui des erreurs. Il suffit d'une petite rayure sur un disque, d'une perturbation de l'appareillage, ou d'un quelconque phénomène parasite pour que le message transmis comporte des erreurs, c'est-à-dire des « 0 » qui ont malencontreusement été changés en « 1 », ou inversement. Or l'un des nombreux atouts du numérique est la possibilité de détecter, et même de corriger, de telles erreurs.

1. Proposez une solution de base pour détecter une erreur.

2. En critiquant forces et faiblesse de cette méthode, proposez une solution qui permette de détecter mais également de corriger l'erreur émise.

3. Illustrez votre explication en codant la lettre « j » qui se traduit par 106 dans la table

ASCII.

4. Dans quel cas, le message demanderait-il une retransmission ?

Table des matières

1 Traitement formel, de quoi s'agit-il ?....4
1.1 Traitement formel et informel de l'information....4
1.2 Exercice pratique : Traitements formels et informels....6
1.3 Traitement formel de l'information : idées maîtresses....7
1.4 Exercice pratique : la communication par code est universelle....9
1.5 La communication par code idéale....11
2 Le codage des nombres....12
2.1 La base décimale, binaire ou hexadécimale (conversions de bases)....12
Conversion décimal - binaire....12
2.2 Autres bases (conversions de bases)....13
2.3 Représentation des nombres entiers....14
Représentation d'un entier naturel....14
Représentation d'un entier relatif....14
Principe du complément à deux....14
Exercice 1 :....15
Exercice 2 :....15
2.4 Représentation des nombres réels....16
Conversion de binaire en décimal....16
Conversion de décimal en binaire....16
Conversion hexadécimal - binaire....18
Exercice 1....18
Exercice 2....18
Exercice 3....18
2.5 Le codage des images....19
3 Codes détecteurs et correcteurs d'erreurs....21
Pourquoi ces techniques de détection d'erreurs et de correction codage existent-elles?
....21
3.2 Schéma général de communication....22
3.3 Principe général :....22
3.4 Exemples de la vie courante....24
3.5 Distance de Hamming....25
Exemples :....25
3.6 Exercice de synthèse....26